

字句解析器を作る — 黒箱を開ける（前編）

今日のゴール

`scaffold/lexer.py` を読んで実際の字句解析器の構造を理解し、同じ仕様の字句解析器を自作する。最後に、講義で使ってきた全テスト入力（約 100 本の `.c` ファイル）で、自作 `lexer` のトークン列が `scaffold` と完全一致することを `golden test` で確認する。

1 この回の位置づけ

- フロントエンド発展シリーズの第 1 回（選択制）。前提は F0（CYK の回）
- F0 の `tokenize` は数式用の 5 種類のトークンだけだった。この回では、それを **Core** プロファイルの **C** の字句仕様に拡張する
- 編集するのは `mylexer.py` だけ。`scaffold` 本体は変更しない

2 Core プロファイルのトークン

`scaffold/lexer.py` は、トークンを次の 7 種類に分類している。

種別	意味	値の入る場所	例
<code>TK_NUM</code>	整数リテラル	<code>val</code>	<code>42</code>
<code>TK_CHAR</code>	文字リテラル	<code>val</code> （文字コード）	<code>'a'</code> → 97
<code>TK_STR</code>	文字列リテラル	<code>sval</code> （実体）	<code>"hi"</code>
<code>TK_IDENT</code>	識別子	<code>sval</code>	<code>main, x1</code>
<code>TK_KW</code>	キーワード	<code>sval</code>	<code>int, return</code>
<code>TK_PUNCT</code>	演算子・区切り	<code>sval</code>	<code>+, <=, ;</code>
<code>TK_EOF</code>	入力終端	—	—

トークンは `Token` dataclass で表され、`kind / val / sval / line`（行番号）を持つ。

たとえば `if (x <= 42)` は次のトークン列になる。

トークン	kind	sval / val
<code>if</code>	<code>TK_KW</code>	<code>sval='if'</code>
<code>(</code>	<code>TK_PUNCT</code>	<code>sval='('</code>
<code>x</code>	<code>TK_IDENT</code>	<code>sval='x'</code>
<code><=</code>	<code>TK_PUNCT</code>	<code>sval='<='</code>
<code>42</code>	<code>TK_NUM</code>	<code>val=42</code>
<code>)</code>	<code>TK_PUNCT</code>	<code>sval=')'</code>

3 scaffold/lexer.py を読む

まず本物を読む。`scaffold/lexer.py` は約 230 行で、前処理器 (`preprocess`) と字句解析器 (`tokenize`) の 2 つからなる。この回で読むのは `tokenize` 側である。

```
less scaffold/lexer.py
```

読解の手がかりとして、`tokenize()` の構造を対応表にしておく。

場所	内容
冒頭の定数	キーワード表・2 文字演算子表・1 文字記号の集合
<code>Token dataclass</code>	トークンの入れ物 (<code>kind / val / sval / line</code>)
<code>while i < n:</code>	メインループ。1 文字見て分岐する (F0 の <code>tokenize</code> と同じ骨格)
空白・改行	読み飛ばす。改行で <code>line</code> を数える
<code>//</code> と <code>/* */</code>	コメントを読み飛ばす
数字	続く限り読んで <code>TK_NUM</code>
<code>'</code> と <code>"</code>	文字・文字列リテラル。エスケープを解決する

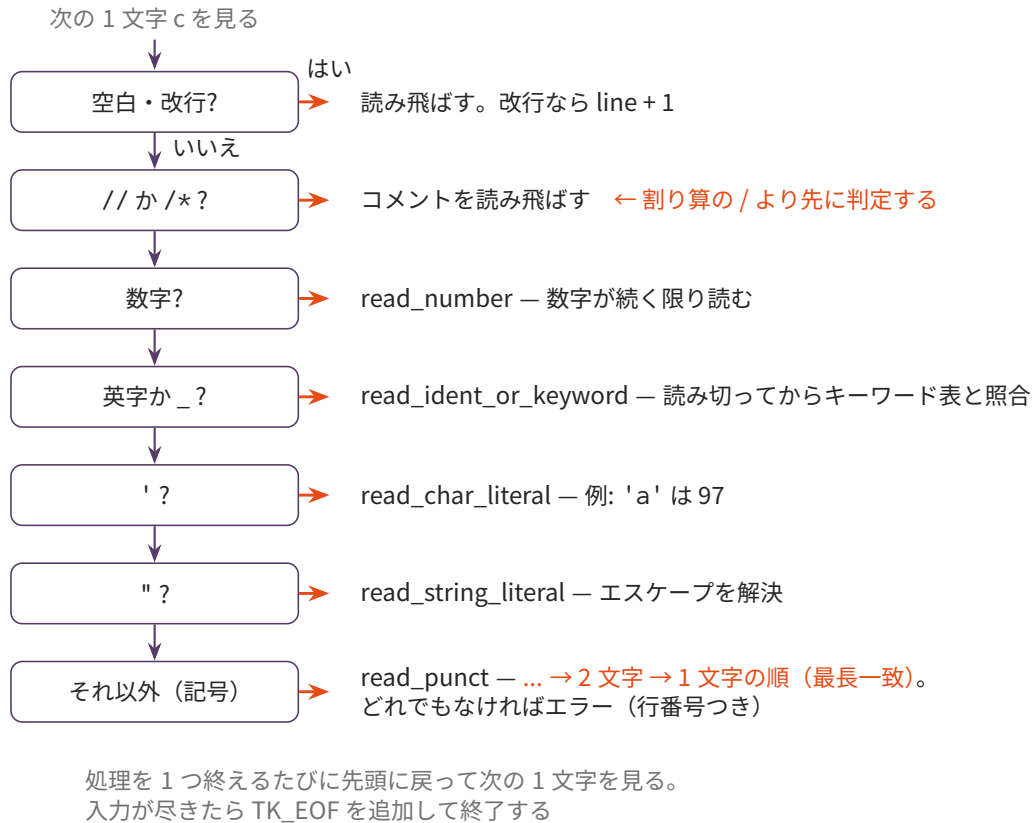


図 1 メインループのディスパッチ (判定は上から順に試す)

場所	内容
英字・ <code>_</code>	続く限り読んで、キーワード表と照合
<code>...</code> → 2 文字 → 1 文字	記号。長い候補から試す
最後	<code>TK_EOF</code> を追加

4 分岐の順序に意味がある

この図の判定の並び順は、入れ替えられない。代表例が `/` である。

入力	正しい解釈
<code>a / b</code>	割り算の <code>TK_PUNCT</code>

入力	正しい解釈
<code>// comment</code>	行コメント（トークンにしない）
<code>/* ... */</code>	ブロックコメント（トークンにしない）

同じ `/` で始まる 3 つを区別するには、コメントの判定を記号の判定より先に行うしかない。順序を逆にすると、`// comment` が「割り算・割り算・識別子 `comment`」に化けてしまう。

記号どうしにも順序がある。`<=` を `<` と `=` に分けないために、長い候補（... → 2 文字 → 1 文字）から試す。これは F0 で数字を読んだときと同じ最長一致の原則である。

補足

`a-->b` は何になるか。最長一致で読むと `a - -> b` になる（`--` は Core プロファイルにないため）。本物の C では `--` があるので `a -- > b` となり、意味が変わる。字句解析の規則が変わると、同じ文字列でも違うトークン列になる例である。

5 キーワードは「読み切ってから」決める

`if` はキーワードだが、`ifx` は識別子である。1 文字ずつ見ながら「`i` の次が `f` だからキーワードだ」と早合点すると `ifx` を壊してしまう。

正しい手順は次の通り。

1. 英数字と `_` が続く限り読み切る（最長一致）
2. 読んだ単語がキーワード表（KEYWORDS）にあれば `TK_KW`、なければ `TK_IDENT`

「まず貪欲に読み、あとで分類する」— これも最長一致の応用である。

6 リテラルとエスケープ

文字リテラル `'a'` は、文字コードを値に持つ `TK_CHAR` になる（`val=97`）。コマ 11 で見たとおり、C では文字は小さな整数だからである。

エスケープは表引きで文字コードに変換する。

書き方	文字コード	意味
<code>'\n'</code>	10	改行
<code>'\t'</code>	9	タブ
<code>'\0'</code>	0	NUL
<code>'\\'</code>	92	バックスラッシュ
<code>'\'' / '\\"'</code>	39 / 34	引用符

文字列リテラル `"x=%d\n"` は、エスケープを解決した実体を `sval` に持つ。`sval` の中身は 6 文字 (`x = % d 改行`) であり、ソース上の `\` と `n` の 2 文字はこの時点で 1 文字に畳まれている。コマ 11 の `.data` 出力で `.byte 10` を出せたのは、Lexer がここで変換を済ませていたからである。

7 行番号 —エラー報告の生命線

Token の `line` は、エラーメッセージの「`main.c:12:`」の材料になる。改行を数える場所は 1 箇所ではない。

改行が現れうる場所	数える処理
トークンの間の空白	<code>skip_whitespace</code>
ブロックコメントの中	<code>skip_comment</code>
文字列リテラルの中	<code>read_string_literal</code>

どれか 1 つでも数え忘れると、それ以降の全トークンの行番号がずれる。`golden test` は行番号まで比較するので、ずれがあれば検出できる。

8 実装

`mylexer.py` のスケルトンは `Lexer` クラスになっている。メインループ（ディスパッチ）と

共通部品（`error / startswith / add / escape_char`）は完成済みで、読み取りメソッド 7 つが TODO である。

Step	実装対象	内容
1	<code>skip_whitespace /</code> <code>skip_comment / read_number</code>	空白・コメント・数。F0 の <code>tokenize</code> + コメント
2	<code>read_ident_or_keyword</code>	読み切ってからキーワード表と照合
3	<code>read_punct</code>	... → 2 文字 → 1 文字 の順に試す
4	<code>read_char_literal /</code> <code>read_string_literal</code>	エスケープの解決
5	（新規実装なし）	行番号の総点検。Step 1 ～4 の数え漏れを直す

各メソッドの方針は `docstring` に書いてある。`scaffold` との違いは 1 点だけ: エラーは `exit` ではなく `SyntaxError` 例外にしている（テストしやすくするため）。

途中経過は `main()` で目視できる。

```
python3 mylexer.py ../../../../sessions/03_arithmetic_codegen/tests/add.c
```

9 テスト

9.1 単体テスト（Step ごと）

```
python3 check.py
```

Step 1～4 は (`kind`, `val`, `sval`) を比べ、Step 5 で行番号まで比べる。未実装の Step は SKIP と表示される。

9.2 golden test (scaffold との突き合わせ)

```
python3 golden.py
```

`sessions/*/tests/*.c` と `final/tests/*.c` の全ファイル (約 100 本) について、`scaffold` と自作 `lexer` のトークン列を `(kind, val, sval, line)` で完全比較する。`#include` / `#define` を含むファイルは、`scaffold` の `preprocess` を両者に通して条件を揃えてある (前処理は F1 の対象外)。

全ファイル **PASS** がこの回の完了条件である。これが通れば、講義で使ってきた黒箱の前半分は、もう黒箱ではない。

補足

golden test という考え方。「正しい出力を人手で書く」代わりに「信頼できる既存実装と突き合わせる」テスト手法を **golden test** (または差分テスト) という。仕様書を書くより安く、テストケースは既存の入力を流用できる。コンパイラ開発では、`gcc` と自作コンパイラの実行結果を突き合わせる形でも使われる (セルフホスト検証もこの発想である)。

10 発展課題

1. 終端しないブロックコメントの扱い: `"1 /* comment"` を `scaffold` に食わせると何が起きるか観察する。実は `scaffold` はエラーを出さない (仕様の穴)。自作版ではエラーにしてよい — `check.py` はエラーになることを期待している
2. エラーメッセージの改善: 該当行のテキストを表示し、問題の位置に `^` を付ける
3. 16 進リテラル: `0x2a` を読めるように拡張する (`scaffold` にない拡張なので、`golden test` は対象外にして単体テストで確認する)
4. 計測: 一番大きいテスト入力で `scaffold` と自作版の実行時間を比べる

補足

コラム: 手書きループは DFA である。「今どの状態にいて、次の 1 文字で何をするか」だけで動くこの種のループは、理論的には**決定性有限オートマトン (DFA)** と等価である。トークンの形 (識別子・数・記号) が**正規言語**で書けるからこそ、先読みほぼなしの一直線のループで字句解析ができる。`flex` などの `lexer` 生成系は、正規表現から DFA を自動生成してこのループを作っている。手書きした今なら、生成系が何を自動化しているのか

が分かるはずである。

11 次回予告

トークン列は手に入った。次の F2 では、いよいよ再帰下降構文解析に入る。

F0 の CYK 法は「どんな文法でも $O(n^3)$ 」だったが、再帰下降は**文法の各規則を 1 つの関数に対応させ**、先読み 1 トークンで一直線に木を組み立てる。`language_spec.md` の EBNF がそのまま関数の設計図になる。まず式（優先順位 10 段）から始める。